

Operating Systems

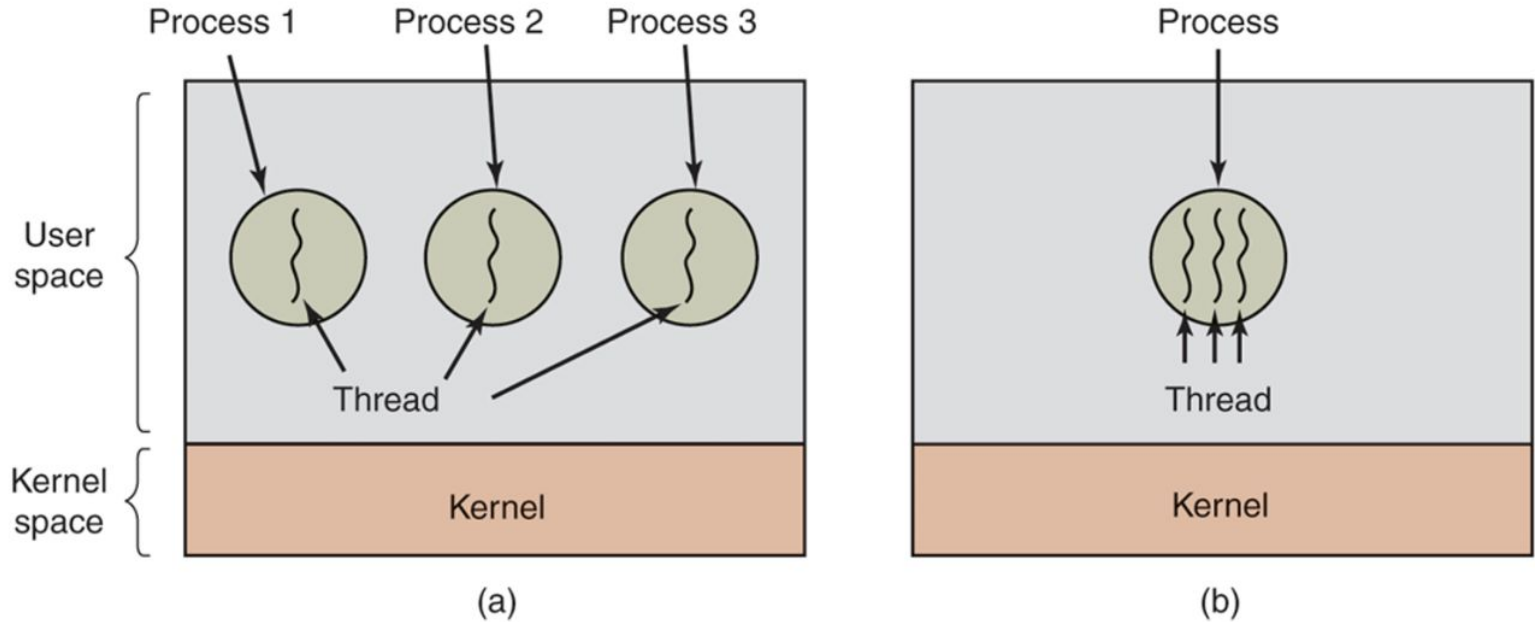
Rowan University

Overview

1. Interprocess Communication Review
2. Scheduler
3. `resched`

Process & Thread Model

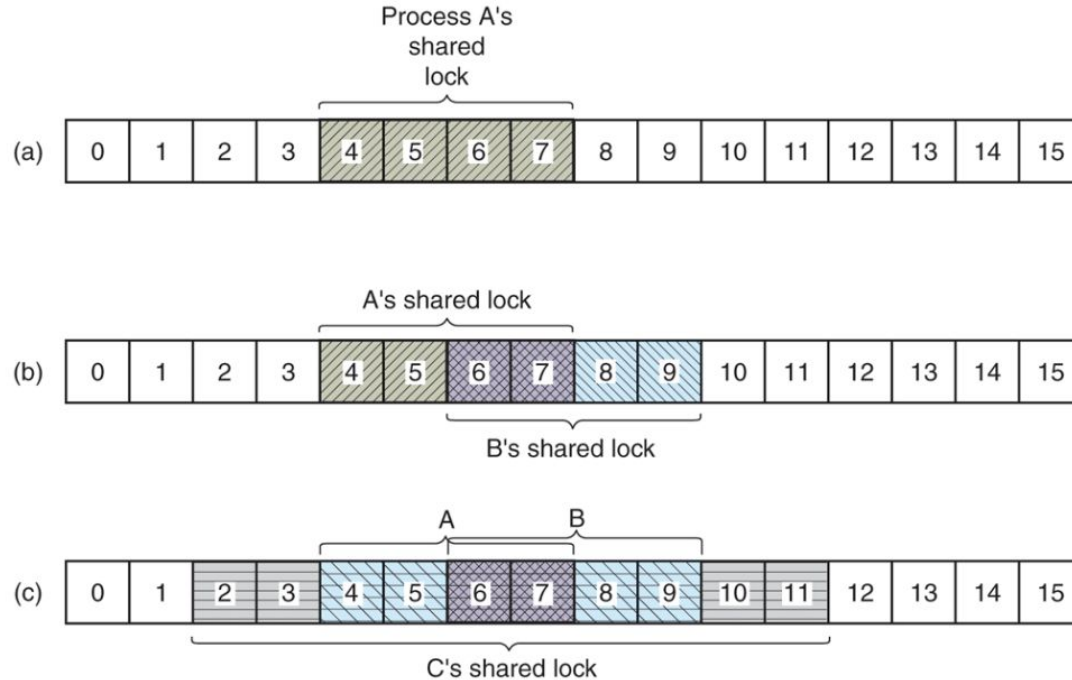
Figure 2-10



(a) Three processes each with one thread. (b) One process with three threads.

Linux File Locking

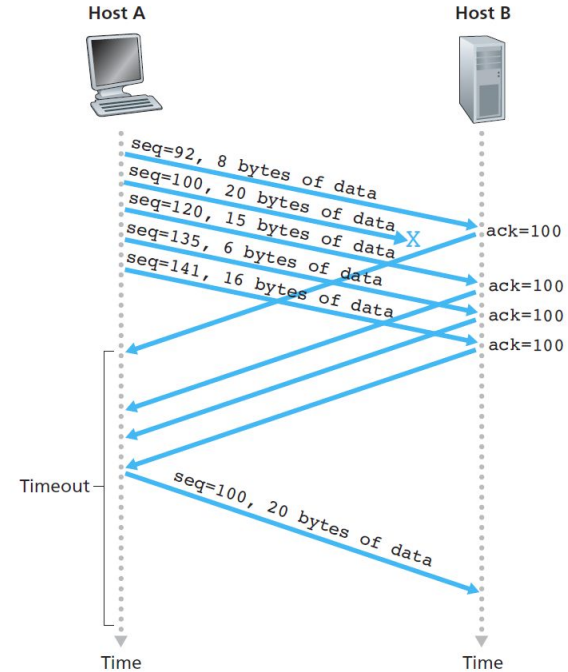
Figure 10-26



(a) A file with one lock. (b) Adding a second lock. (c) A third one.

Message Passing

- Message Passing: Interprocess communication using system calls
- Primitives: `send` and `receive`
- Design Issues in Message-Passing Systems:
 - Challenges in distributed systems
 - Lost messages: Agreement on acknowledgment
 - Handling message retransmission with sequence numbers
 - Naming processes and authentication



Semaphores

- Similar to a binary mutex lock except it acts as a queue of waiting processes
- Semaphore is an integer
 - Equal to the number of process waiting to be woken up
- Process sleeps when semaphore is 0 and `down` is called on it
 - `down(&full)`

Figure 2-28

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

```
/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */
```

```
/* TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of full slots */
```

```
/* infinite loop */
/* decrement full count */
/* enter critical region */
/* take item from buffer */
/* leave critical region */
/* increment count of empty slots */
/* do something with the item */
```

Interprocess Communication (IPC)

Issues to consider:

1. Processes need to **communicate** with each other
2. Processes need to **avoid clashing** into each other
3. Processes need to **handle dependencies** on each other

Communication and clashing stem from the dependency on shared memory

Scheduling Processes

- Multi-programmed computers handle multiple processes concurrently
- The **scheduler** decides which process or thread to execute next
- Scheduling algorithm determines which processes to run

- Key Concepts
 - **Scheduler:** Part of the OS responsible for selecting the next process or thread
 - **Scheduling Algorithm:** Determines the criteria for selecting the next process or thread

- Common Issues
 - Resource contention: Multiple processes competing for CPU time
 - Efficiency: Maximizing CPU utilization
 - Fairness: Ensuring equitable access to CPU resources

Scheduling Processes

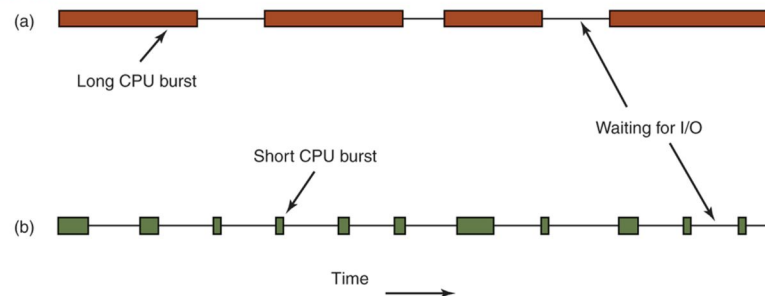
Efficiency Concerns:

- *Process switching* is costly in terms of CPU time
- **Context switch** involves moving between user and kernel mode
- Saving and reloading process state, memory maps, and MMU configurations
- Too many switches per second can degrade performance significantly

Process Behavior

- Processes alternate between computing and I/O operations
- Compute-bound: Long CPU bursts, infrequent I/O waits
- I/O-bound: Short CPU bursts, frequent I/O waits
- Impact of CPU Speed
 - CPU speeds increase faster than I/O speeds (e.g. disk read/write speed)
- Scheduling of I/O-bound processes may become more critical

Figure 2-40



Bursts of CPU usage alternate with periods of waiting for I/O. (a) A CPU-bound process. (b) An I/O-bound process.

Key Scheduling Events

- **Process Creation:**
 - Decision between parent and child process
 - Scheduler can choose either
- **Process Exit:**
 - Selection of new process from ready processes
 - If no process is ready, system idle process runs
- **Process Blocking:**
 - Selection of new process when one blocks on I/O or semaphore
 - Dependency considerations often overlooked by scheduler
- **I/O Interrupt:**
 - Decision when an I/O device completes work
 - Scheduler chooses among newly ready, current, or other processes

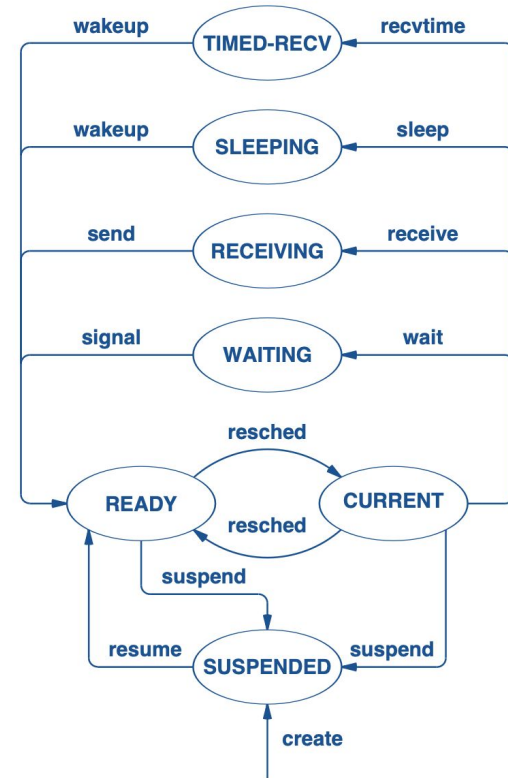


Figure 13.3 State transitions including the *timed receive* state.

Preemptive Scheduling

- Clock Interrupts
 - Periodic interrupts trigger scheduling decisions
- Nonpreemptive Scheduling:
 - Process runs until block or voluntary release
 - No scheduling decisions during clock interrupts
 - Higher-priority process may preempt
- Preemptive Scheduling:
 - Process runs for fixed time interval
 - Suspended if not completed; another process chosen
 - Requires clock interrupts to return control to scheduler

```
/* Decrement the preemption counter, and reschedule when the */  
/* remaining time reaches zero */  
  
if(--preempt) <= 0) {  
    preempt = QUANTUM;  
    resched();  
}
```

Scheduling Algorithms

- **Batch Systems:**
 - Used in business for periodic tasks
 - No impatient users waiting at terminals
 - Nonpreemptive or long-time preemptive algorithms acceptable
- **Interactive Environments:**
 - Preemption essential to prevent CPU hogging
 - Servers fall into this category
- **Real-time Systems:**
 - Preemption sometimes not needed
 - Processes quickly do their work and block
 - Processes know they may not run for long periods

Figure 2-41

All systems

Fairness—giving each process a fair share of the CPU
Policy enforcement—seeing that stated policy is carried out
Balance—keeping all parts of the system busy

Batch systems

Throughput—maximize jobs per hour
Turnaround time—minimize time between submission and termination
CPU utilization—keep the CPU busy all the time

Interactive systems

Response time—respond to requests quickly
Proportionality—meet users' expectations

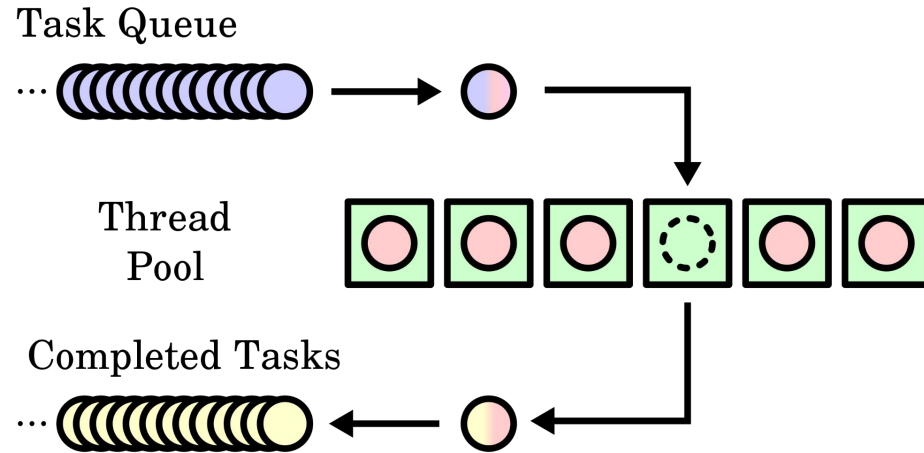
Real-time systems

Meeting deadlines—avoid losing data
Predictability—avoid quality degradation in multimedia systems

Some goals of the scheduling algorithm under different circumstances.

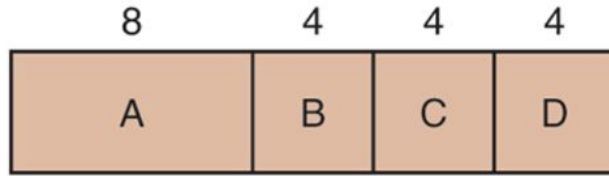
First Come First Serve

- CPU assigned to processes in the order they request it
- Algorithm:
 - Single queue of ready processes
 - New jobs added to end of queue
 - Running process not interrupted until it blocks
- Strength: Very simple to implement
- Weakness: CPU-bound process can monopolize CPU, delaying I/O-bound processes

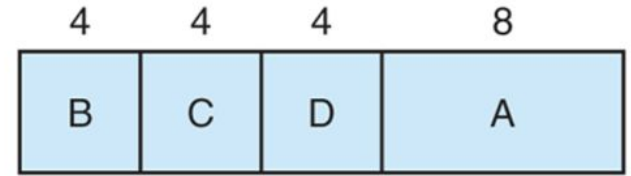


Shortest Job First

Figure 2-42



(a)



(b)

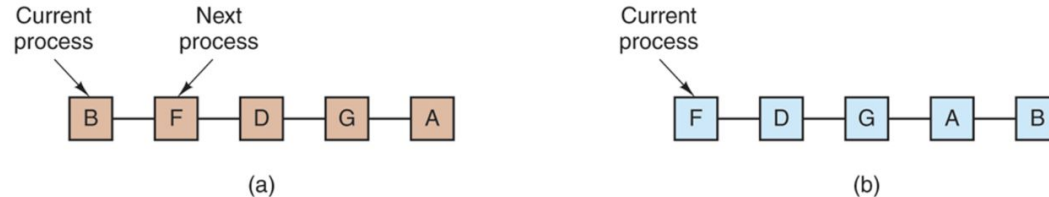
An example of shortest-job-first scheduling. (a) Running four jobs in the original order. (b) Running them in shortest-job-first order.

Round Robin Scheduler

- Each process is assigned a fixed time **quantum**
- If process doesn't finish within its quantum, it's preempted
- 10 - 100 milliseconds

- Algorithm Overview:
 - Maintain list of runnable processes
 - Process at front runs until quantum ends or it blocks
 - If blocked, next process runs; if not, back of list

Figure 2-43

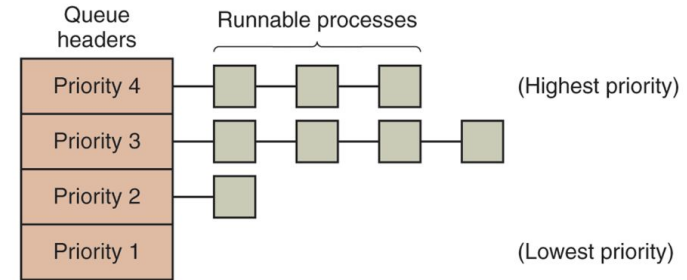


Round-robin scheduling. (a) The list of runnable processes. (b) The list of runnable processes after *B* uses up its quantum.

Priority Scheduling

- Assigns priority to each process
- Highest priority process runs first
 - Scheduler may decrease the priority of the currently running process at each clock tick
- Differentiates among users and system processes
- Static: Predefined priorities based on user or system criteria
- Dynamic: Adjusted based on process behavior or system goals

Figure 2-44

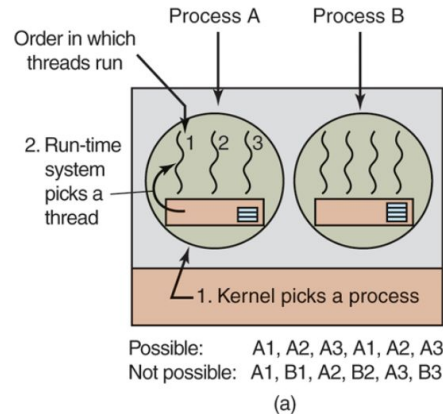


A scheduling algorithm with four priority classes.

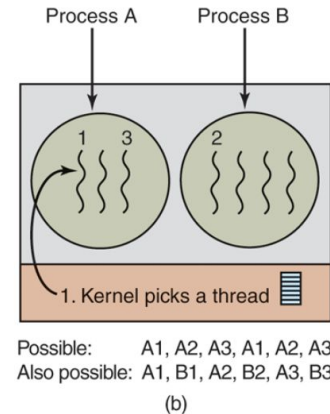
Thread Scheduling

- User-Level Threads:
 - Kernel unaware of threads.
 - Process quantum allocated
 - Thread scheduler manages thread execution

Figure 2-45



- Kernel-Level Threads:
 - Kernel manages thread scheduling
 - Thread suspension upon quantum expiration
 - Allows thread switches between processes



(a) Possible scheduling of user-level threads with a 50-msec process quantum and threads that run 5 msec per CPU burst. (b) Possible scheduling of kernel-level threads with the same characteristics as (a).

Xinu Unified Structure for Linked Lists of Processes

- Xinu Process Manager (scheduler) uses several lists
- To accommodate all cases, a representation was selected with the following properties:
 - All lists are doubly linked, which means a node points to its predecessor and successor
 - Each node stores a key as well as a process ID, even though a key is not used in a FIFO list
 - Each list has head and tail nodes; the head and tail nodes have the same memory layout as other nodes
 - Non-FIFO lists are ordered in descending order; the key in a head node is greater than the maximum valid key value, and the key value in the tail node is less than the minimum valid key

Xinu Unified Structure for Linked Lists of Processes

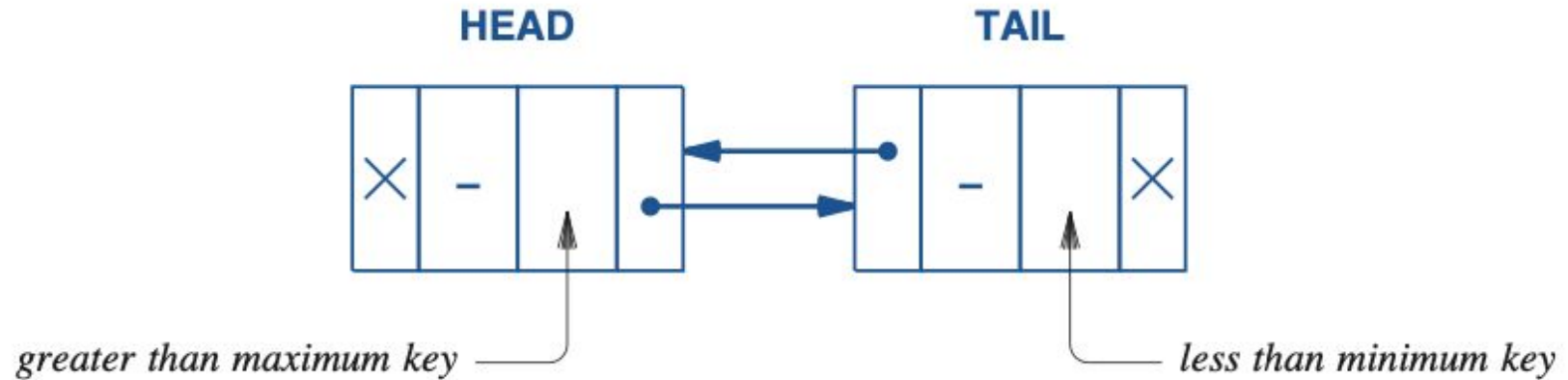


Figure 4.2 The conceptual form of an empty linked list.

Xinu Unified Structure for Linked Lists of Processes

Figure 4.1 illustrates the conceptual organization of a linked list data structure by showing an example list with two items.

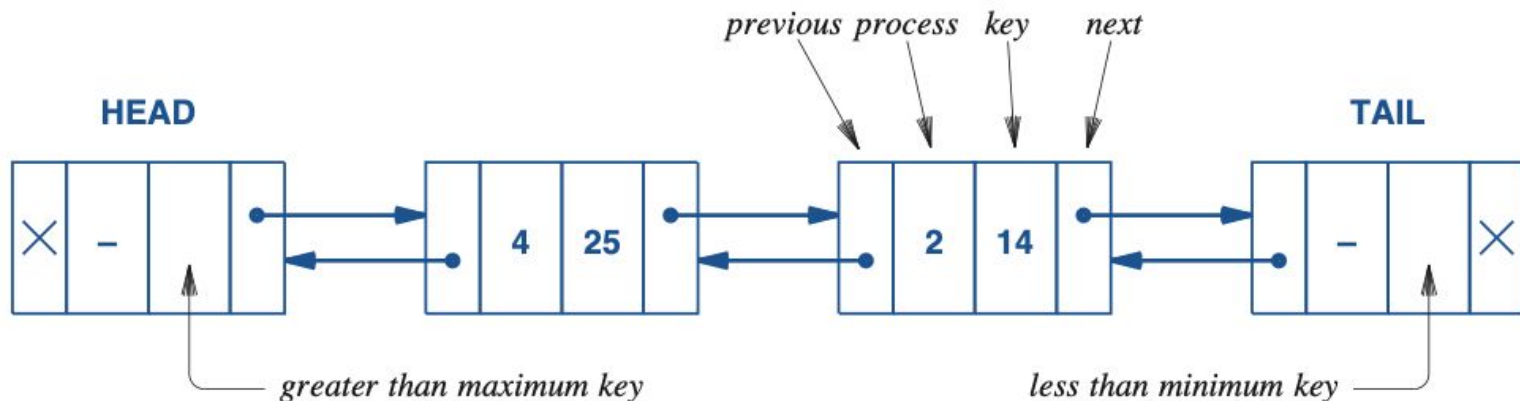


Figure 4.1 The conceptual organization of a doubly-linked list containing processes 4 and 2 with keys 25 and 14, respectively.